

Reviewer Pack — Minimal Order of Quadratic Residues in Tseitin Formulas Boun...

Entry ffb7a91f1ecc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 08:09:52 UTC

Minimal Order of Quadratic Residues in Tseitin Formulas Bounds Resolution Proof Width

Entry ID: ffb7a91f1ecc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 08:09:52 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Residues) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

For a given Tseitin formula φ with n variables, the minimal order of quadratic residues among its clauses is bounded by the resolution proof width of φ , specifically, $O(\sqrt{n})$. Equivalently, for any instance φ , if the minimal order q of quadratic residues in φ 's clauses is such that $q = \Theta(\sqrt{n})$, then the resolution proof width $w(\varphi)$ satisfies $w(\varphi) = O(q)$.

Rationale (proposer's reasoning):

Quadratic residues provide a way to encode boolean functions algebraically. This conjecture posits a potential connection between algebraic encoding and the complexity of finding proofs, which could expose new structural insights into resolution complexity.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 46461bc0100bcf7e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal order of quadratic residues (q) and resolution proof width (w) for at least 30% of the Tseitin formulas is greater than or equal to 0.7, with $q = \Theta(\sqrt{n})$. The criterion is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_tseitin_formula(n):
    variables = list(range(1, n + 1))
    clauses = []

    for var in variables:
        clauses.append([var])

    tseitin_vars = [n + i + 1 for i in range(n)]

    for i in range(n):
        clauses.append([-variables[i], -tseitin_vars[i]])
        clauses.append([variables[i], tseitin_vars[i]])
        for j in range(i + 1, n):
            clauses.append([-tseitin_vars[i], -tseitin_vars[j]])
            clauses.append([tseitin_vars[i], tseitin_vars[j]])

    return variables, clauses

def minimal_order_of_quadratic_residues(clauses):
    residues = set()
    for clause in clauses:
        for lit in clause:
            if lit > 0:
                residues.add(lit)
            else:
                residues.add(-lit)
    min_order = float('inf')
    for r in residues:
        order = 1
        while (r ** order) % n != 1 and order <= n:
            order += 1
        if order < min_order:
            min_order = order
    return min_order
```

```

def resolution_width(clauses):
    # Simplified version of resolution width calculation
    # This is a placeholder and should be replaced with actual implementation
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [10, 20, 30, 40]
    results = []

    for n in n_values:
        variables, clauses = generate_tseitin_formula(n)
        q = minimal_order_of_quadratic_residues(clauses)
        w = resolution_width(clauses)

        results.append({
            "n": n,
            "q": q,
            "w": w
        })

    total_q = sum(result["q"] for result in results)
    total_w = sum(result["w"] for result in results)
    mean_q = total_q / len(results)
    mean_w = total_w / len(results)

    correlation_coefficient = (sum((result["q"] - mean_q) * (result["w"] - mean_w) for result in results) /
                             math.sqrt(sum((result["q"] - mean_q) ** 2 for result in results) *
                                           sum((result["w"] - mean_w) ** 2 for result in results)))

    conjecture_holds = correlation_coefficient >= 0.7
    counterexample = "" if conjecture_holds else f"Correlation coefficient {correlation_coefficient}"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if not trial_result["conjecture_holds"]:
            break

```

```

        results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = seeds[results.index(next(result for result in results if not result["conjecture_holds"]))]
    print(f"RESULT: FALSIFIED counterexample='Correlation coefficient < 0.7' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1670552c.py", line 103, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1670552c.py", line 66, in run_trial
    q = minimal_order_of_quadratic_residues(clauses)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1670552c.py", line 47, in minimal_order_of_quadratic_residues
    while (r ** order) % n != 1 and order <= n:
           ^
NameError: name 'n' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient between the minimal order of quadratic residues and resolution proof width. | next: Investigate the error in the test code to ensure it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16288
2	propose	ollama_remote	glm4:latest	0	0	14095
3	preregistration	ollama_remote	glm4:latest	0	0	16097
4	novelty	ollama_remote	glm4:latest	0	0	10095
5	novelty	ollama_remote	glm4:latest	0	0	9042
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12968
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12861
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14680
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13006
10	judge	ollama_remote	glm4:latest	0	0	14614

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133746 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ffb7a91flecc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ffb7a91flecc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ffb7a91flecc.tar.gz` (if generated)